

AI Agents Cheat Sheet

19 tables · 144 concepts · Updated 2026-05-28 · [cheatgrid.com](#)

AI agents are autonomous systems built on large language models that can **perceive their environment, reason through complex tasks, and take actions** using external tools to achieve goals. Unlike traditional chatbots that simply respond to queries, agents operate through continuous **think-act-observe loops**, dynamically planning their next steps based on outcomes. The defining characteristic is **tool use**—agents don't just generate text; they execute functions, query databases, call APIs, and coordinate with other agents through standardized protocols like **MCP**, **A2A**, and **AG-UI**. This shift from prediction to execution makes agents the foundation of agentic AI, transforming LLMs from assistants into **operational systems** capable of multi-step workflows, self-correction, and long-horizon task completion. Understanding agent architecture—perception, reasoning engines, memory systems, and orchestration patterns—is essential for building reliable production agents in 2026. Frameworks like LangGraph, CrewAI, OpenAI Agents SDK, Google ADK, and PydanticAI provide production-ready primitives, while **context engineering** has emerged as a first-class discipline for controlling what an agent sees at each step, and evaluation tools like DeepEval and LLM-as-Judge enable systematic quality measurement.

CONTENTS

Core Agent Concepts	2
Architecture Patterns	3
Memory Systems	4
Multi-Agent Systems	4
Communication Protocols	5
Agent Frameworks	6
Tool Integration	7
State Management	8
Execution Patterns	9
Reasoning Techniques	9
Planning Strategies	10
Error Handling	10
Evaluation & Testing	11
Observability & Debugging	12
Retrieval-Augmented Generation (RAG) for Agents	12
Context Management	13
Security & Safety	14
Cost Optimization	15
Production Patterns	15
References	16

TABLE 1

Core Agent Concepts

These are the building blocks every AI agent shares regardless of framework: a control loop that thinks-acts-observes, structured tool invocation, a reasoning engine, perception of inputs, and a calibrated level of autonomy. Grasp these and the framework-specific patterns in later tables become small variations on the same theme.

Concept	Example	Description
Agent Loop	<pre>while not done: thought = think(observation) action = decide(thought) observation = execute(action)</pre>	• Continuous think-act-observe cycle where the agent reasons about current state, takes an action, observes the result, then decides the next step • the foundational execution pattern popularized by ReAct, where each iteration depends on the latest observation rather than a fixed plan.
Tool Use	<pre>tools = [calculator, web_search, db_query] agent.invoke("What's 2^16?", tools)</pre>	• Broad pattern where the agent invokes external functions or APIs beyond text generation, with the runtime (not the LLM) executing the call and feeding the result back • tools are defined with name, description, and JSON schema for parameters.
Function Calling	<pre>{"name": "get_weather", "args": {"city": "NYC"}}</pre>	• LLM-side mechanism that outputs structured JSON specifying which function to call and with what arguments • the model only proposes the call; the application or agent runtime is what actually executes it and returns the result.
Agentic Workflow	Goal → Plan → Execute → Observe → Adjust → Repeat	• Dynamic, goal-driven process where the agent decomposes the task, takes actions, evaluates outcomes, and adapts its strategy • distinct from a linear prompt chain, which follows fixed predefined steps and is a workflow, not an agent.
Perception	<pre>raw_input = {"text": query, "context": session} structured_data = parse(raw_input)</pre>	• Transforms raw inputs (text, API responses, sensor data) into structured representations the reasoning engine can act on • includes parsing, normalization, filtering noise, and entity extraction — more than just receiving text.
Reasoning Engine	LLM + prompting strategy (ReAct/CoT) + memory + planning	• Core decision-making component that weighs inputs, selects actions, and generates plans for the agent • typically an LLM operating inside a scaffold of memory, planning, and prompting strategies — the agent is the orchestrated system, not the bare LLM.
Action Space	<pre>["send_email", "query_db", "http_get", "file_write"]</pre>	• Set of available tools and operations the agent is allowed to invoke • defines the boundary of what the agent can accomplish in its environment; tools outside this set cannot be selected by the LLM.
Computer-Using Agent (CUA)	<pre>agent.screenshot() agent.click(x=320, y=240) agent.type("search query")</pre>	• Agent uses multimodal vision to interpret a screen then plans and executes keyboard/mouse actions, enabling control of any GUI without a dedicated API • pure vision-based CUAs (OpenAI CUA, Azure Responses API) handle any OS or app; DOM-aware variants (browser-use, UFO) add structural hints for higher accuracy on constrained targets.
Autonomy	Agent decides when to stop vs. asks user for approval per action	• Degree to which the agent makes decisions without human intervention, measured as a spectrum rather than a binary • ranges from fully autonomous to selective human-in-the-loop (HITL) gates on high-stakes actions — the dominant production pattern in 2026.

TABLE 2

Architecture Patterns

These patterns describe how a single agent system organizes reasoning, decision-making, and control flow. The biggest source of confusion is what happens WHEN: ReAct decides one step at a time, Plan-and-Execute commits to a full plan upfront, Reflection critiques its own output, and Tree-of-Thought explores branches in parallel before committing.

Pattern	Example	Description
ReAct	Thought: Need price data Action: <code>query_db("SELECT price")</code> Observation: \$42 Thought: Now calculate...	• Reasoning and Acting interleaved • agent generates Thought explaining reasoning, executes Action, receives Observation of result, then generates next Thought • think-act-observe pattern improves reliability and debuggability.
Reflection	<code>output = generate()</code> <code>critique = evaluate(output, criteria)</code> <code>if not good: revise(critique)</code>	• Agent self-evaluates outputs against quality criteria, then iterates to improve • generates, critiques, and refines until meeting standards or max attempts.
Evaluator-Optimizer	<code>output = generator.propose(task)</code> <code>feedback = evaluator.critique(output, criteria)</code> <code>if not done:</code> <code>generator.revise(feedback)</code>	• Generator produces output, evaluator critiques against criteria, loop repeats until quality threshold met • enables autonomous quality assurance via two cooperating agents instead of one.
Planning	Goal → Subtasks → Order → Execute	• Agent decomposes complex goals into actionable subtasks, determines execution order, then coordinates completion • enables long-horizon task solving.
Chain-of-Thought	Let's think step-by-step: 1. Parse question 2. Identify data needed 3. Calculate result	• Prompts agent to show intermediate reasoning steps • improves accuracy on complex tasks by making thought process explicit before answering.
Router	<code>if query_type == "sql":</code> <code>route_to(sql_agent)</code> <code>else:</code> <code>route_to(general_agent)</code>	• Conditional dispatcher that directs requests to specialized agents or tools based on content analysis • enables modular, expert-based architectures.
Orchestrator-Worker	<code>supervisor.assign(task, worker_agents)</code> <code>results = await all_workers()</code> <code>supervisor.synthesize(results)</code>	• Hierarchical delegation where supervisor breaks work into parallel subtasks, assigns to worker agents, aggregates results • scales complex workflows efficiently.
Agent Handoff	<code>handoff(source=triage_agent,</code> <code>target=billing_agent,</code> <code>condition="billing question")</code>	• One agent transfers control to another specialized agent mid-conversation • enables seamless delegation where the receiving agent continues with full context.
Plan-and-Execute	<code>plan = planner.create(goal)</code> <code>for step in plan:</code> <code>execute(step)</code>	• Separates planning from execution • planner creates full task breakdown upfront, then executor runs each step • clearer than pure ReAct for multi-step workflows.
Tree-of-Thought	Explore multiple reasoning paths in tree structure, backtrack if needed	• Agent explores multiple solution paths simultaneously, evaluating each branch before committing • useful when single linear reasoning path may fail.

TABLE 3

Memory Systems

Memory turns stateless LLMs into agents that learn across sessions. This table maps the canonical short-term vs long-term split, the three cognitive long-term types (episodic, semantic, procedural), production runtimes like Mem0, Letta (MemGPT) and Zep/Graphiti, and the working/shared memory layers that coordinate state within and across agents.

Type	Example	Description
Short-Term Memory	Current conversation context in prompt	• Thread-scoped context for the active session • holds recent messages and intermediate outputs inside the model's context window; cleared when the thread ends.
Long-Term Memory	Vector DB storing past interactions	• Persistent storage across threads and sessions • agent retrieves relevant historical context to inform current decisions • critical for maintaining user preferences and learning.
Mem0	<pre>mem0.add(messages, user_id="u1") results = mem0.search("user prefs", user_id="u1")</pre>	• Dedicated memory layer that dynamically extracts, consolidates, and retrieves salient facts from conversations across sessions • achieves 26% relative improvement over OpenAI's memory and 91% lower p95 latency vs full-context; integrates with 21 frameworks and 20 vector stores.
Zep/Graphiti	<pre>graphiti.add_episode(messages) results = graphiti.search("user prefs", center_node_uuid)</pre>	• Temporal knowledge graph memory storing facts as edges with bi-temporal validity windows • outperforms vector-only approaches on cross-session temporal reasoning (94.8% vs 93.4% on DMR; up to 18.5% gain on LongMemEval).
Episodic Memory	"On 2026-01-15, user preferred JSON output"	• Stores specific past events tied to a time and place • lets the agent recall "what happened when" instead of only "what is true in general."
Semantic Memory	"User always wants reports in PDF format"	• Stores durable facts, preferences, and learned knowledge that are true in general • extracted and consolidated from episodic traces into atomic, deduplicated entries.
Procedural Memory	Learned workflows like "how to file a bug report"	• Encodes how to perform tasks — communication styles, formatting rules, action sequences • captured from feedback and reused as the agent's default behavior.
Letta (MemGPT)	<pre>agent = create_agent(model="openai/gpt-4o") agent.send_message("Prefer JSON output")</pre>	• OS-inspired virtual context management with tiered memory (core in-context blocks, recall, archival storage) • the agent self-edits its own memory blocks via tool calls (<code>memory_insert</code> , <code>memory_replace</code>).
Working Memory	Variables tracking current task state	• Active scratchpad for the current task • holds intermediate results, loop counters, and task-specific variables that the agent manipulates while reasoning — distinct from raw conversation history.
Shared Memory	<pre>redis.set("team_context", state) other_agent.get("team_context")</pre>	• Cross-agent state enabling coordination • multiple agents read/write a common store (Redis, shared DB) so they stay aligned in multi-agent systems.

TABLE 4

Multi-Agent Systems

This table covers the common topologies multi-agent systems use to coordinate work — when control is centralized, when it isn't, and how agents share state. The split between hierarchical (a supervisor delegates downward) and network/collaborative (peers interact as equals) is the most-confused distinction, and how state is shared (direct messages vs a shared blackboard) is the second.

Pattern	Example	Description
Hierarchical	<code>supervisor → specialist_a, specialist_b → workers</code>	• A central supervisor delegates tasks to specialized agents, which may themselves supervise sub-teams • communication and routing flow through the supervisor, results aggregate upward • mirrors an org chart and makes ownership and debugging easier than a flat mesh.
Agent-as-Tool	<code>orchestrator.tools = [agent_a.as_tool(), agent_b.as_tool()]</code>	• A sub-agent is wrapped as a callable tool so an orchestrating agent invokes it via standard function calling • unlike handoffs (which transfer control), agent-as-tool invocations return a result to the calling agent, which retains control; preferred when the caller needs the sub-result before deciding its next step.
Collaborative	Agents share a scratchpad of messages and discuss until one says <code>FINAL ANSWER</code>	• Peer agents interact as equals to solve a problem jointly, with no fixed authority • they exchange messages, debate, and negotiate solutions until they reach consensus.
Blackboard	<code>board.write("draft", text)</code> then a reviewer agent triggers when it sees a new draft and writes back <code>board.write("review", feedback)</code>	• Agents read and write to a shared workspace rather than messaging each other directly • each agent watches the board and activates when relevant data appears, enabling loose coupling and emergent coordination without a central orchestrator.
Sequential	<code>Agent A → Agent B → Agent C</code> (pipeline)	• Agents are chained in a predefined linear order, each processing the previous agent's output • suited to step-by-step workflows where each stage depends on the one before it.
Parallel (Concurrent)	Three agents analyze the same stock concurrently, then a final step merges their answers	• Multiple agents work on independent subtasks at the same time, with no agent reading another's in-progress output • a coordinator aggregates results when they finish, cutting overall latency for decomposable work.
Network	Any agent can call any other agent's tool directly; routing is decided at runtime	• A decentralized many-to-many mesh where any agent can communicate with any other and decide which agent runs next • no fixed hierarchy or predefined order — flexible but harder to debug than supervised topologies.

TABLE 5
Communication Protocols

Agents rarely work alone — they reach out to tools and data, hand off work to other agents, and stream results back to users. This table covers the three protocols that now define those boundaries (**MCP**, **A2A**, **AG-UI**) along with the classic messaging patterns (**Pub-Sub**, **Request-Response**, **Message Queue**) that still underpin agent transports.

Protocol	Example	Description
Model Context Protocol (MCP)	<code>mcp_server.list_tools()</code> <code>mcp_server.call_tool("get_data", args)</code>	• Open standard originated by Anthropic (November 2024) for connecting LLM applications to external tools, resources, and prompts • uses a host–client–server architecture over JSON-RPC 2.0 with Streamable HTTP transport (HTTP+SSE deprecated since the March 2025 spec revision) • the canonical agent-to-tools/data layer, distinct from agent-to-agent.
Agent-to-Agent (A2A)	<code>agent_a.send(agent_b, message)</code> <code>response = agent_b.process_and_reply()</code>	• Inter-agent communication protocol originated by Google, now donated to the Linux Foundation • agents publish agent cards at <code>/.well-known/agent-card.json</code> and exchange task-lifecycle messages to delegate work • IBM's ACP (Agent Communication Protocol) was officially incorporated into A2A under the Linux Foundation in August 2025.

Protocol	Example	Description
AG-UI	<pre>agent.emit(TextMessageStart(id)) agent.emit(ToolCallStart(id, name))</pre>	• Open, event-based protocol for the agent ↔ user-interface boundary, born from CopilotKit's work with LangGraph and CrewAI • streams text chunks, tool calls, state updates, and human-in-the-loop events over SSE or WebSocket • natively supported by Amazon Bedrock AgentCore Runtime, LangGraph, CrewAI, Microsoft Agent Framework, Google ADK, and others.
Pub-Sub	<pre>agent.subscribe("topic/events") publish("topic/events", data)</pre>	• One-to-many broadcast pattern — every subscriber to a topic receives its own copy of each message • publishers don't know how many subscribers exist, which decouples senders from receivers • natural fit for event-driven fan-out across many agents.
Request-Response	<pre>response = await agent.call(request)</pre>	• Synchronous query-reply pattern — the caller blocks until the callee returns a response • simplest model with strong consistency and easy debugging, but produces tight runtime coupling and risks cascading failures • the baseline pattern HTTP / REST inherits.
Message Queue	<pre>queue.push(task) worker = queue.pop() worker.execute(task)</pre>	• Point-to-point asynchronous delivery — each message is consumed by exactly one worker, with FIFO ordering inside the queue • decouples producers from consumers and buffers spikes, with built-in retry and dead-letter handling • the workhorse pattern for background work distribution (e.g. SQS, RabbitMQ queues).

TABLE 6

Agent Frameworks

The agent-framework landscape in 2026 is crowded, and each tool below picks a different bet: graph-based stateful runtimes, role-based crews, conversational multi-agent systems, type-safe structured output, model-driven SDKs, or lightweight code-first harnesses. Use this table to match a project's needs (durability, model-agnosticism, multi-agent style) to the framework whose design philosophy fits best.

Framework	Example	Description
LangGraph	<code>StateGraph</code> with nodes, edges, and checkpoints	• Graph-based orchestration for stateful, cyclical workflows • models agents as state machines with conditional routing and durable persistence via checkpoints • production-grade.
LangChain	<code>create_agent(model, tools=[...])</code> with chains, tools, memory	• Flexible toolkit for building LLM applications • provides abstractions for prompts, tools, memory, and a <code>create_agent</code> entry point • code-first with extensive integrations.
CrewAI	<code>Crew</code> of agents with <code>role</code> , <code>goal</code> , <code>backstory</code>	• Role-based collaboration where agents simulate team dynamics • supports <code>Process.sequential</code> and <code>Process.hierarchical</code> (requires a <code>manager_llm</code> or <code>manager_agent</code>) • fast prototyping for multi-agent workflows.
OpenAI Agents SDK	<code>Agent(name="Assistant", tools=[...])</code> <code>Runner.run_sync(agent, query)</code>	• Official OpenAI framework, the production-ready replacement for the experimental Swarm library • core primitives: Agents, Handoffs, Guardrails, and Agent-as-Tool patterns (plus Sessions and Tracing) • Python-first with built-in tracing and MCP support.
AutoGen	<code>ConversableAgent</code> with multi-agent conversations	• Conversational agents that communicate via message passing • emphasizes agent-to-agent dialogue and group chats for task solving • Microsoft-backed.
Claude Agent SDK	<code>async for msg in query(prompt="Fix the bug", options=ClaudeAgentOptions())</code>	• Anthropic's official SDK exposing the same agent harness that powers Claude Code as a library • built-in tools for file reading, command execution, and web search • available in Python and TypeScript.

Framework	Example	Description
Google ADK	<code>SequentialAgent</code> , <code>ParallelAgent</code> , <code>LoopAgent</code> , <code>LlmAgent</code>	• Google's modular framework with workflow agents (Sequential, Parallel, Loop) for deterministic flow and <code>LlmAgent</code> for LLM-driven dynamic routing • model-agnostic • multi-language (Python, Go, Java, TypeScript).
PydanticAI	<code>agent = Agent('openai:gpt-5.2', output_type=MyModel)</code>	• Type-safe Python framework by the Pydantic team, FastAPI-like for GenAI • structured output with automatic Pydantic validation, MCP/A2A/AG-UI integration • built-in evals and Logfire observability.
Semantic Kernel	<code>kernel.add_plugin(MyPlugin(), plugin_name="X")</code> plus planners	• Microsoft framework optimised for enterprise scenarios • tight Azure integration, C# / Python / Java support • emphasises plugins as reusable skills the kernel and planners can invoke.
Strands Agents	<code>agent = Agent(model=BedrockModel(...), tools=[...])</code>	• AWS open-source SDK with a model-driven approach • model-agnostic, supporting Bedrock, Anthropic, OpenAI, Gemini, and Ollama with one-line provider swaps • native MCP support, tool hot-reloading, and a Swarm multi-agent pattern.
smolagents	<code>agent = CodeAgent(tools=[tool], model=model)</code> <code>agent.run("What is the weather?")</code>	• Hugging Face's lightweight agent library built around code-first actions • <code>CodeAgent</code> writes and executes Python snippets as actions (must be sandboxed); <code>ToolCallingAgent</code> is the JSON-tool-call alternative • supports local and remote models.
Agno	<code>agent = Agent(model=OpenAIChat(), tools=[...])</code> <code>agent.print_response("Summarize this")</code>	• Fast, lightweight framework for building multi-modal agents • supports memory, knowledge, reasoning, and teams • headline claim: agent instantiation on the order of microseconds, far faster than heavier frameworks.
Claw Code	<code>git clone github.com/ultraworkers/claw-code</code> <code>./target/debug/claw prompt "Refactor auth"</code>	• Open-source Rust-based CLI agent harness (April 2026) inspired by Claude Code • multi-provider support (Anthropic, xAI, OpenAI-compatible, DashScope) with a tiered permission system and session persistence • build-from-source only (the <code>cargo install</code> stub is deprecated).

TABLE 7

Tool Integration

Function calling tells you *that* an LLM can ask for a tool — this table covers the runtime plumbing that actually makes tools work. It maps the lifecycle from defining the schema, discovering what's available, executing the call, parsing the result, and scheduling multiple calls in parallel or in sequence, including the efficient code-execution pattern for large MCP server ecosystems.

Technique	Example	Description
Tool Schema	<code>{"name": "get_weather", "description": "...", "parameters": {...}}</code>	• JSON-Schema description of the tool's name, when to use it, and the shape of its parameters • the model reads this metadata to decide <i>whether</i> and <i>how</i> to invoke the tool — implementation code lives in your runtime, never in the schema.
Tool Discovery	<code>tools = mcp_client.list_tools()</code>	• Runtime enumeration of available tools via <code>tools/list</code> over JSON-RPC • lets agents pick up new capabilities without redeployment, but every advertised schema costs context tokens.
Tool Execution	<code>name, args = parse(tool_call)</code> <code>result = tools[name](**args)</code>	• Agent runtime invokes the function the LLM selected, parsing the call name and JSON arguments • the LLM only generates the call; if your code skips dispatch, the model fabricates an observation instead.
Structured Output	<code>response_format={"type": "json_schema", "strict": true, "schema": {...}}</code>	• Constrained decoding masks any token that would violate the supplied JSON Schema, guaranteeing schema-conformant output • stronger than the older JSON mode (which only guarantees valid JSON syntax) • supported by OpenAI, Anthropic, Google, and AWS Bedrock; guarantees shape, not semantic correctness.

Technique	Example	Description
Parallel Tool Use	<pre>calls = [get_weather("NYC"), get_weather("LA")] results = await asyncio.gather(*calls)</pre>	- The model emits multiple tool calls in one turn, and the runtime dispatches them concurrently - works only when calls share no data dependency — hidden dependencies (e.g. fetch-then-update on shared state) create race conditions.
Tool Chaining	<pre>user_id = get_user(name) orders = get_orders(user_id)</pre>	Sequential composition where the output of tool A feeds the input of tool B - the data dependency forces serial execution and is the structural opposite of parallel tool use.
Tool Result Parsing	<pre>obs = clean(tool_output) messages.append({"role": "tool", "content": obs})</pre>	Converts raw tool output into a clean observation message the LLM can reason over - typically JSON-stringifies structured data and strips noise like rate-limit headers, request IDs, and other internal metadata.
Code Execution via MCP	<pre>import * as gdrive from './servers/google-drive' const doc = await gdrive.get Document({documentId})</pre>	- Presents MCP servers as code APIs on a filesystem rather than direct tool calls — the agent reads only the tool definitions it needs and processes data in a code execution environment before returning results to the model - reduces token usage by up to 98.7% by avoiding upfront loading of all tool schemas and keeping intermediate results out of the model context window.

TABLE 8

State Management

State management is what lets an AI agent survive crashes, hand off conversations between sessions, and explore alternative trajectories without losing the original run. The patterns below cover the two dominant approaches in production today — LangGraph's checkpointer-based persistence and Temporal's event-sourced durable execution — and the supporting concepts (threads, typed schemas, rollback) that make either approach safe at scale.

Concept	Example	Description
Checkpointing	<pre>graph.compile(checkpointer=PostgresSaver(...)) graph.get_state(config)</pre>	Saving agent state as a snapshot at every super-step boundary - enables pause/resume, time-travel debugging, and recovery from node failures - critical for long-running agents.
State Persistence	<pre>PostgresSaver.from_conn_string(DB_URI) SqliteSaver, RedisSaver</pre>	Durable storage of agent state across process restarts - production agents swap <code>InMemorySaver</code> for Postgres/SQLite/Redis backends so threads survive crashes and redeploys.
Thread Management	<pre>config = {"configurable": {"thread_id": "user-123"}} graph.invoke(input, config)</pre>	Isolating parallel agent sessions <code>thread_id</code> is the primary key under which checkpoints are stored, giving each user or session independent state with no crosstalk.
State Schema	<pre>class AgentState(TypedDict): messages: Annotated[list, add_messages] step_count: int</pre>	Typed definition of agent state structure (TypedDict, dataclass, or Pydantic BaseModel) - reducers like <code>add_messages</code> tell LangGraph how to merge updates — without them, the last write wins.
Durable Execution	<pre>@workflow.defn class AgentWorkflow: async def run(self): ...</pre>	- Agent workflows that survive crashes and restarts via platforms like Temporal - recovery works by replaying the recorded Event History against deterministic workflow code — not by loading a saved state blob.
State Rollback	<pre>fork_cfg = graph.update_state(old_cfg, {"x": "new"}) graph.invoke(None, fork_cfg)</pre>	Forking from an earlier checkpoint with modified state to explore an alternative path <code>update_state</code> is non-destructive — the original history is preserved, enabling safe "undo" and experimentation.

TABLE 9

Execution Patterns

How an agent is invoked controls its throughput, perceived latency, and integration shape. These five patterns — synchronous, asynchronous, streaming, event-driven, and batch — solve different problems and frequently get confused with one another, especially async vs parallel and streaming vs async.

Pattern	Example	Description
Synchronous Execution	<pre>result = agent.run(query) print(result)</pre>	• Blocking call that waits for agent completion before returning • simpler to reason about but locks the caller until the full response is ready.
Asynchronous Execution	<pre>task = asyncio.create_task(agent.run(query)) result = await task</pre>	• Non-blocking invocation that yields control while the agent works • concurrency (single-threaded event loop), not parallel execution; lets the caller interleave other I/O.
Streaming	<pre>async for token in agent.stream(): print(token, end="")</pre>	• Agent emits partial outputs (token deltas via SSE) as they are generated • cuts Time-to-First-Token and improves perceived latency without changing total generation time.
Event-Driven	<pre>agent.on("tool_call", log_callback) agent.on("error", retry_callback)</pre>	• Callback-based execution where typed events (run lifecycle, tool calls, state deltas, errors) fire registered handlers • canonical pattern in the AG-UI protocol for agent-UI integration.
Batch Processing	<pre>batch = client.messages.batches.create(requests=[...]) # poll, then read results</pre>	• Submit many requests in one job processed asynchronously over up to 24 hours • 50% cheaper on OpenAI and Anthropic; suited to evals, classification, bulk generation.

TABLE 10

Reasoning Techniques

Reasoning techniques shape **how** an agent thinks through a problem before answering — from solving with no examples (zero-shot) to ensembling many sampled reasoning paths (self-consistency) to looping with the environment (agentic reasoning). The right choice depends on the task's complexity, the cost budget, and whether the agent needs to ground itself in external observations.

Technique	Example	Description
Zero-Shot	<pre>"Translate this to French: Hello"</pre>	• Agent solves the task with no examples in the prompt, relying solely on instruction and pre-training • no weight updates — distinct from fine-tuning • fastest and cheapest, but less reliable for complex or ambiguous tasks.
Few-Shot	<pre>Examples: Q: 2+2 A: 4 Q: 3+5 A: 8 Now: 7+9 = ?</pre>	• A handful of input-output example pairs are placed in the prompt before the actual query • the model conditions on these demonstrations in-context — no gradient updates (Brown et al., GPT-3, 2020) • teaches task format and output style purely through demonstration.
Self-Consistency	<pre>Run the same CoT prompt 5 times with temperature > 0, return the majority answer</pre>	• Samples multiple independent chain-of-thought paths for the same question, then returns the answer that appears most often (Wang et al., 2022) • replaces greedy decoding with sample-and-vote aggregation • boosts hard-reasoning accuracy (e.g. +17.9% on GSM8K) at roughly N× the token cost.
Graph-of-Thought	<pre>Nodes = LLM thoughts, edges = dependencies; aggregate, refine, loop</pre>	• Generalizes Tree-of-Thought to an arbitrary graph of thoughts (Besta et al., 2023), enabling aggregation across branches, cycles, and feedback-loop refinement • unlike a tree, separate reasoning paths can be merged into a synergistic thought • trades higher orchestration cost for quality gains on elaborate problems.

Technique	Example	Description
Agentic Reasoning	Thought → Action → Observation → Thought → ... (ReAct loop)	• Closed-loop reasoning where the agent thinks, takes an action (tool call, API), observes the outcome, and adjusts the next thought • canonical instantiation is ReAct (Yao et al., 2022) — interleaved thought-action-observation traces • grounds the agent in the environment, which sharply reduces the fact-hallucination that pure chain-of-thought suffers from.

TABLE 11

Planning Strategies

Five strategies an agent uses to turn a goal into action. They differ in **when** the plan is built, **how** it is structured, and **what** the agent does when reality fails to cooperate — from a one-shot decomposition into subtasks, through multi-level hierarchies, to runtime replanning and the upfront Planner-Worker-Solver split that ReWOO introduces.

Strategy	Example	Description
Task Decomposition	"Write report" → ["research", "outline", "draft", "edit"]	• Breaking complex goal into subtasks • agent identifies logical steps required to achieve objective, each simpler than original.
Hierarchical Planning	High-level plan → Detailed sub-plans for each step	• Multi-level decomposition where agent plans at multiple granularities • top-level strategy refined into tactical execution steps.
Dynamic Replanning	Adjust plan when action fails or new info appears	• Agent updates strategy based on execution results • abandons unsuccessful paths and generates new plans in response to changing conditions.
Contingency Planning	If primary approach fails, execute backup plan	• Creating alternative strategies upfront • agent has predefined fallbacks for anticipated failure modes.
ReWOO	Planner generates full tool-use plan upfront without intermediate observations	• Reasoning WithOut Observation — separates planning from tool execution • planner creates complete action sequence before any tool is called, reducing redundant LLM calls • more token-efficient than ReAct for predictable workflows.

TABLE 12

Error Handling

Production agents fail constantly: providers throttle, networks blip, models time out, and downstream tools return garbage. These five patterns are the standard distributed-systems toolkit applied to LLM agents — retry transient errors with backoff and jitter, fall back to alternatives, trip a circuit breaker before a retry storm crashes a recovering provider, degrade gracefully when full output is impossible, and propagate unrecoverable errors up to a supervisor.

Technique	Example	Description
Retry with Backoff	<pre>for attempt in range(3): try: call_api() except: sleep(2**attempt)</pre>	• Automatic retry with exponentially increasing delays • handles transient failures like rate limits or network glitches.
Fallback Strategies	<pre>try: use_gpt4() except: use_gpt35()</pre>	• Alternative approaches when primary fails • agent switches to backup model, tool, or method if first choice unavailable.
Circuit Breaker	After N failures, stop trying for cooldown period	• Prevents cascading failures • temporarily disables failing service to allow recovery rather than overwhelming it with retries.
Graceful Degradation	Return partial results when full task impossible	• Agent completes what it can even when encountering errors • provides best-effort output rather than total failure.
Error Propagation	Pass error context upward in multi-agent hierarchy	• Bubbles failures to supervisor agents who can make recovery decisions • maintains error visibility while delegating handling.

TABLE 13

Evaluation & Testing

Metrics, frameworks, and benchmark suites that production teams use to answer two distinct questions about an agent: *did it succeed?* and *did it succeed for the right reasons?* Outcome-style measures (Task Success Rate, SWE-bench, GAIA) sit alongside trajectory-style measures (Trajectory Analysis, Tool Accuracy) and the human-in-the-loop and LLM-as-Judge graders that score everything in between.

Metric	Example	Description
Task Success Rate	<code>successful_tasks / total_tasks</code>	• Headline outcome metric — fraction of evaluation tasks the agent completes against the success criteria • lets you compare agent versions and benchmark sizes apples-to-apples.
Trajectory Analysis	Evaluate the reasoning path and tool calls, not just the final answer	• Inspects the full transcript: reasoning steps, tool calls, intermediate state • catches agents that pass via lucky paths and reveals <i>why</i> failures happen, not just <i>that</i> they happened.
Tool Accuracy	<code>correct_tool_calls / total_tool_calls</code>	• Action-layer metric: did the agent select the right tools with the right arguments? • foundational for tool-using agents — poor tool selection cascades into everything else.
Hallucination Rate	<code>fabricated_facts / total_statements</code>	• Frequency of invented information not supported by the provided context • measured against ground truth or a retrieval context; lower is better.
LLM-as-Judge	<code>judge_llm.score(output, rubric)</code>	• Use a stronger LLM to grade outputs against a rubric or pick a winner in a pairwise comparison • scales human evaluation but inherits judge biases like position, verbosity, and self-enhancement.
SWE-bench	Resolve real GitHub issues from popular Python repos	• Standard benchmark for coding agents — an agent passes only if its patch makes the hidden test suite go from failing to passing • SWE-bench Verified is near-saturated in 2026 (top agents ~100%); SWE-bench Pro on Scale AI's SEAL leaderboard is the new standard (multi-language, harder harness — same top agent drops to ~46%) • data contamination on Verified led OpenAI to stop reporting those scores.
GAIA	466 real-world tasks mixing web browsing, file parsing, multi-document reasoning; top agents ~75% in 2026	• General AI Assistants benchmark that chains tool use, web browsing, and reasoning across three difficulty levels • progress from ~20% (2023) to 74.5% in early 2026; the same model scores up to 7 points differently across orchestration frameworks — the scaffold, not just the model, determines results.
τ-bench	Agent chats with LLM-simulated user AND calls tool APIs; pass ^k measures reliability across k re-runs	• Real-world conversational agent benchmark with domain-specific policies (retail and airline) testing multi-turn interaction, tool use, and rule-following • measures reliability via pass^k — top models drop from ~45–71% on pass¹ to ~25% on pass⁸, revealing production unreliability hidden by single-run averages.
DeepEval	<code>assert_test(test_case, [ToolCorrectnessMetric()])</code>	• Open-source, pytest-style LLM evaluation framework (Apache 2.0) • ships deterministic metrics (Tool Correctness) alongside LLM-as-Judge metrics (G-Eval, Hallucination, RAGAS, Task Completion).
HAL (Holistic Agent Leaderboard)	Princeton's standardized, cost-aware leaderboard covering GAIA, SWE-bench, WebArena, TAU-bench, and more	• Unified cost-aware evaluation harness (accepted ICLR 2026) for reproducible comparison across benchmarks and agent frameworks • tracks cost-performance Pareto frontier; agents can be 100× more expensive for only 1% accuracy gain — a one-dimensional leaderboard hides this.
Human Feedback	Expert review of agent transcripts or user satisfaction ratings	• Gold-standard grading for subjective qualities like helpfulness, tone, and edge-case judgment • expensive and slow, so often reserved for calibrating LLM judges and spot-checking.
Benchmark Datasets	AgentBench (OS/DB/web tasks), WebArena (web automation), MMLU (knowledge), HumanEval (code)	• Standardised public test sets that enable apples-to-apples comparison across models • each measures something specific — one benchmark alone never proves general capability; WebArena success rates rose from 15% (2023) to 74.3% (2026); saturated benchmarks lose signal.

TABLE 14

Observability & Debugging

Without observability, multi-step agent failures are nearly un-debuggable: each LLM call, tool invocation, and sub-agent decision happens behind the model's reasoning, and a flat log won't tell you which step caused the wrong answer. This table covers the layered toolkit teams reach for in production — tracing for causal execution trees, logging and real-time monitoring for what's happening now, dedicated platforms like LangSmith, Langfuse, and Laminar, callback hooks for low-overhead instrumentation, and replay for reproducing intermittent bugs from saved state.

Tool	Example	Description
Tracing	<pre>@traceable def assistant(q): ...</pre>	• Captures the execution tree as parent-child spans across every LLM call, tool use, and sub-agent step • reveals causality and timing that flat logs cannot.
Logging	<pre>logger.info(f"Agent chose tool: {tool_name}")</pre>	• Time-stamped records of discrete events written to a persistent store • useful for post-mortem analysis and compliance auditing, but lacks span-level causality.
Real-Time Monitoring	Dashboard showing trace count, latency p50/p99, error rate, cost	• Live production visibility with prebuilt panels for traces, LLM calls, tools, and costs • threshold alerts fire when error rate or latency cross configured limits.
LangSmith	<pre>export LANGSMITH_TRACING=true # traces auto-captured</pre>	• LangChain's hosted observability platform with LangGraph Studio IDE for visual agent debugging, 1-click deployment, and zero-setup tracing for LangChain/LangGraph apps • full OTel support as of March 2026; self-hosting is Enterprise only.
Langfuse	<pre>@observe(as_type="agent") def run_agent(q): ...</pre>	• Open-source LLM engineering platform (MIT) from an independent team, not LangChain • framework-agnostic via OpenTelemetry with first-class free self-hosting; strong evaluation, prompt management, and dataset workflows.
Laminar	<pre>@observe(name="agent") async def run_agent(q): ... # session replay synced to trace</pre>	• Real-time agent debugging platform built around a span-tree causal model and first-class Replay workflows • browser-agent session replay is synced to traces — useful for debugging what a CUA actually saw; native OTel ingestion and data-volume pricing distinguish it from LangSmith and Langfuse.
Callback Handlers	<pre>on_llm_start , on_tool_end , on_error</pre>	• Observer-only event hooks triggered at lifecycle points without modifying chain logic • attach via <code>RunnableConfig</code> for per-request scoping.
Replay	<pre>graph.invoke(None, checkpoint_config)</pre>	• Re-executes nodes from a saved checkpoint to reproduce a bug or test a fix • in LangGraph, <code>get_state_history</code> lists checkpoints you can replay or fork from.

TABLE 15

Retrieval-Augmented Generation (RAG) for Agents

RAG techniques connect an agent to external knowledge so its answers can be grounded in fresh, domain-specific facts instead of memorized weights. The patterns below stack: query transformation reshapes the input, vector or graph retrieval pulls candidates, reranking sharpens them, and an agentic controller decides when any of this is even worth running.

Technique	Example	Description
Vector Search	<pre>embeddings = embed(query) results = vector_db.search(embeddings, k=5)</pre>	• Semantic retrieval of relevant documents using embedding similarity (e.g. cosine, dot product) • agent queries a vector store to augment reasoning with external facts.
Agentic RAG	Agent decides when to retrieve, what to query, how to use results	• Agent controls retrieval rather than always fetching upfront • reasons step-by-step about necessity, formulates queries, may iterate or skip retrieval entirely for simple questions.

Technique	Example	Description
Query Transformation	Original query → hypothetical answer (HyDE) → embed and retrieve against that	- Pre-retrieval step that rewrites or expands the query to close the query–document vocabulary gap - includes HyDE, multi-query, and step-back prompting.
Reranking	<pre>candidates = retrieve(query, k=20) top_results = reranker.rank(candidates, k=5)</pre>	- Post-retrieval step that re-scores candidates using a cross-encoder (or LLM judge) that sees query and document jointly - lifts precision after high-recall vector retrieval.
Graph RAG	Query a knowledge graph for entity relationships and community summaries	Retrieves structured knowledge from a graph of entity nodes and relationship edges - supports multi-hop reasoning that flat vector similarity cannot, with substantial gains in answer comprehensiveness on global questions (Microsoft Research, 2024).

TABLE 16

Context Management

Every agent runs on a finite token budget — what you feed the model, what it has said so far, what it retrieved, and what it generates all share the same window. This table covers the techniques that decide what stays, what gets compressed, and what gets cached so an agent can run for hours without the model losing focus or burning your budget on the same tokens twice.

Technique	Example	Description
Context Window	Claude Opus 4.6 / Sonnet 4.6: 1M tokens (GA); Gemini 2.5 Pro: 1M tokens	All tokens the model can reference in one call, including the response it generates - covers system prompt, history, retrieved docs, current query, and output 1M-token windows are now generally available from Anthropic (Opus 4.6 / Sonnet 4.6) and Google.
Context Overflow Handling	Stop with <code>model_context_window_exceeded</code> , then summarize and continue	Strategies for the hard token limit - modern APIs return an explicit error or stop reason at the boundary rather than silently dropping tokens - common responses are truncation, summarization (compaction), or splitting the work across calls.
Prompt Caching	<code>cache_control: {type: "ephemeral"}</code> on a static system prompt	Reuses a static prompt prefix across requests by storing its processed tokens server-side - cache hits require an exact (hash-level) match of the prefix up to the breakpoint - default TTL is 5 minutes (Anthropic); cache reads are billed at a fraction of fresh input tokens; in production delivers ~90% cost reduction on cached tokens.
Semantic Caching	Embed query; if cosine similarity to a stored query exceeds the threshold, return the cached answer	Reuses a prior response when a new query is semantically close to one already answered - matches by embedding similarity, not by exact text — so the model is skipped entirely on a hit - research shows ~31% of LLM queries exhibit semantic similarity; lives in the application layer, distinct from provider-side prompt caching.
Prompt Compression	LLMLingua drops low-information tokens to shrink a prompt up to ~20x	Cuts tokens while preserving meaning for the model - LLMLingua-family methods use a small language model to score and drop low-information tokens at the token level - distinct from summarization, which paraphrases rather than removes tokens.
Dynamic Context Selection	"Just-in-time" loading: agent reads <code>file_paths</code> , queries a DB, or calls a tool only when needed	Agent decides what to load for the current step instead of stuffing everything up front - keeps the active window small and task-focused, avoiding context rot from irrelevant tokens - Anthropic's Claude Code uses this pattern with <code>grep</code>, <code>head</code>, and stored references.

TABLE 17

Security & Safety

Agentic systems amplify traditional LLM risks because the model can act, hold credentials, and chain tool calls — so a single manipulated prompt or poisoned tool description can escalate into data exfiltration or destructive action. The defenses below come from the OWASP Top 10 for Agentic Applications (2026), OWASP's AI Agent Security Cheat Sheet, and current vendor guardrail toolkits; they work as layered controls, not silver bullets.

Technique	Example	Description
Prompt Injection	Hidden instructions in a retrieved document override the agent's system prompt	• Untrusted content embeds instructions the model treats as commands • direct (in user input) or indirect (in retrieved docs, tool output, images, emails) • OWASP LLM01 and the root vector for most agent breaches.
Input Validation	Strip known injection patterns and segregate untrusted content with delimiters before the model sees it	• Treat every external string as untrusted — user input, retrieved docs, tool output, email bodies • sanitize, length-limit, and clearly mark data vs. instructions • one layer of defense in depth, never sufficient alone.
Guardrails	<code>nemoguardrails</code> checks input and output flows around every LLM call	• Runtime constraints that wrap the model (input, dialog, retrieval, execution, and output rails) • NeMo Guardrails, Guardrails AI, Llama Guard, Azure Prompt Shields • programmable middleware — not the same as the model's own safety training.
AI Gateway Guardrails	API base URL → AI gateway (Bifrost, Portkey) → model provider	• Enforces safety policies once at the gateway layer for all model traffic — PII redaction, prompt injection defense, content filtering — without modifying individual agent codebases • production pattern: gateway covers OWASP LLM01/02/05/08; application-level rails (NeMo, Guardrails AI) handle conversational scope and excessive agency.
Sandboxing	Run agent-generated code in a gVisor or Firecracker microVM with no host filesystem access	• Isolate agent code execution so a successful exploit cannot reach the host or sensitive data • containers, gVisor user-space kernel, Firecracker/Kata microVMs, WebAssembly • limits blast radius; does not prevent the exploit itself.
Tool Poisoning	Malicious MCP server hides instructions inside a tool description the user never sees	• Attacker-controlled tool metadata (descriptions, schemas) manipulates the agent at registration time • a supply-chain-style indirect prompt injection unique to tool registries like MCP • enables "rug pulls" and cross-server shadowing of trusted tools.
Action Approval	Agent proposes <code>transfer_funds</code> ; execution blocks until a human approves the exact parameters	• Human-in-the-loop gate for high-impact or irreversible actions only • bind approval to actor, tool, target, and parameters with a short expiry • applied selectively so routine low-risk actions are not slowed.
Access Control	<code>if user.role != "admin": deny_tool("delete_db")</code>	• Least-privilege scoping of tool capabilities based on the calling user and session context • per-tool permission lists, read-vs-write splits, scoped tokens • mitigates OWASP LLM06 Excessive Agency.
Non-Human Identity (NHI)	Each agent gets a unique service identity with rotated short-lived credentials and an explicit owner	• Treat every agent as its own identity with creation, rotation, and revocation lifecycle • machine identities now outnumber humans by an order of magnitude or more in many enterprises • Entro's 2025 report found 97% of NHIs hold excessive privileges.
OWASP Agentic Top 10	Goal hijacking, tool misuse, identity abuse, memory poisoning, cascading failures, rogue agents	• Peer-reviewed 2026 risk framework for autonomous agents, published December 2025 • complements the OWASP LLM Top 10 (model-level) with action-level risks • describes a breach progression, not just an isolated checklist.
Agent Governance Toolkit	<code>pip install agent-governance-toolkit</code> ; YAML policy evaluated on every tool call	• Microsoft open-source runtime security layer (April 2026, MIT license) covering all 10 OWASP Agentic risks • stateless policy engine targeting sub-millisecond p99 enforcement • adapters for LangChain, AutoGen, CrewAI, OpenAI Agents SDK, Semantic Kernel.

Technique	Example	Description
Audit Logging	Record agent ID, session, tool, parameters, decision, and outcome for every action	• Tamper-evident activity trail of decisions, tool calls, approvals, and outcomes • redact secrets before writing, log decision metadata for high-risk actions • powers anomaly detection, incident response, and compliance evidence.

TABLE 18

Cost Optimization

Token bills scale faster than features once an agent moves to production, so cost control is a first-class engineering concern. Agents make 3–10× more LLM calls than simple chatbots, and output tokens cost 3–8× more than input tokens across major providers — making cascading model routing and caching the highest-leverage levers available.

Technique	Example	Description
Model Selection	Route classification to Haiku, save Sonnet/Opus for reasoning	• Task-aware model routing • pick the cheapest model that meets the quality bar for each task class, escalating only on low-confidence responses • the price gap between flagship and small models is roughly 15–190× per token, so routing the simple majority away from the flagship dominates other savings.
Cascade Routing	Small model → confidence check → escalate to frontier only if confidence below threshold	• Dynamic confidence-based model escalation — each query is sent to the cheapest model first; only low-confidence or high-entropy responses escalate to a larger model • well-implemented cascades can reduce costs by up to 87% by routing ~90% of queries to small models; OpenAI's GPT-5 architecture uses this pattern internally.
Prompt Caching	Cache a stable system prompt + tools prefix; vary only the user message	• Reusing repeated prompt portions across calls • on Anthropic, cache writes cost 1.25× base input (5-min TTL) or 2× (1-hour TTL), and cache reads cost 0.1× base input (~90% savings on cached tokens) • only pays off above the reuse break-even; a breakpoint on a changing block writes every request and never reads, increasing the bill.
Batch API	Submit overnight classification of 10,000 documents	• Asynchronous batch processing at a 50% discount on both input and output tokens • results returned within 24 hours (often faster) on OpenAI and Anthropic — suitable for evaluations, backfills, and analytics, never for real-time user requests.
Output Token Limits	<code>max_tokens=200</code> for summaries vs <code>max_tokens=2000</code> for essays	• Capping generation length with <code>max_tokens</code> / <code>max_completion_tokens</code> • bounds the most expensive token class (output is 3–8× input price on most models) and prevents pathological long responses; the cap limits only generation, not the input prompt.
Early Stopping	Break the ReAct loop when the final-answer tool fires	• Agent terminates the reasoning loop once the goal is reached rather than burning through a fixed iteration budget • each loop iteration is one LLM call, so stopping early on success and detecting redundant tool loops removes the tail of wasted spend.
LLM FinOps	Tag every call with <code>feature_id</code> + <code>tenant_id</code> ; alert at 50/80/100% of monthly budget	• Applying the financial-operations discipline to AI inference spend • tag-then-aggregate per feature/agent/tenant, then measure cost-per-outcome (resolved ticket, completed task) instead of cost-per-token; without per-agent attribution, kill switches and budgets cannot be enforced.

TABLE 19

Production Patterns

Shipping an agent to production exposes failure modes the demo never showed: retried payments creating duplicate charges, runaway tool loops draining budgets, silent half-completions, and crashed workers losing hours of progress. These five patterns are the safety rails that turn a clever agent into a reliable service — idempotent retries, selective human gates, outcome verification, hard time ceilings, and durable state on shutdown.

Pattern	Example	Description
Idempotency	Idempotency-Key: <uuid> on a retried POST	• Retry safety for write operations • same key returns the cached first result instead of repeating the side effect • critical for payments, emails, and any tool with external consequences.
Human-in-the-Loop	<code>interrupt()</code> pauses graph before sending an email	• Selective approval gates on high-risk or irreversible actions (5–15% of steps) • agent proceeds autonomously until it hits a gated tool, then waits for <code>Command(resume=...)</code>.
Closed-Loop Execution	After a tool call, read back state and verify the change landed	• Agent observes the actual outcome before the next step, not just the API status code • catches "200 OK but row never written" failures and re-plans.
Timeout Management	<code>await asyncio.wait_for(agent(), timeout=60)</code>	• Hard wall-clock ceiling on a single run • cancels the task and raises <code>TimeoutError</code> so runaway loops or hung tool calls can't burn budget forever.
Graceful Shutdown	Persist checkpoint to DynamoDB on SIGTERM, resume on restart	• Preserve work-in-progress by writing state to durable storage before exit • lets the next worker pick up from the last super-step instead of replaying from scratch.

References

Official Documentation

1. OpenAI Agents SDK — Overview
2. OpenAI Assistants API — Function Calling
3. OpenAI Responses API — Computer-Use
4. OpenAI Chat Completions — `max_completion_tokens`
5. Anthropic — Building Effective Agents
6. Anthropic — Model Context Protocol Specification
7. Anthropic — Prompt Caching
8. Anthropic — Creating Message Batches
9. Anthropic — Claude Models Overview
10. Anthropic — Code Execution with MCP
11. LangChain — Agents Concepts
12. LangGraph — Interrupts and Human-in-the-Loop
13. LangGraph — Checkpointing
14. LangGraph — Multi-Agent Architectures
15. LangSmith — Observability Documentation
16. AutoGen — Multi-Agent Conversation Framework
17. Microsoft Semantic Kernel — Agents
18. Microsoft Azure — Computer-Using Agents (CUA)
19. Microsoft Agent Governance Toolkit — Open Source Release
20. Microsoft UFO — Windows Desktop Agent
21. Google DeepMind — AlphaCode 2 Technical Report
22. Google ADK — Agent Development Kit
23. AWS — Durable AI Agents with LangGraph and DynamoDB
24. AWS Strands Agents — Documentation
25. AWS Strands Agents — Swarm Multi-Agent Pattern
26. CrewAI — Documentation
27. Hugging Face — smolagents Documentation
28. Python asyncio — `wait_for`
29. Stripe — Idempotent Requests
30. OWASP — LLM01 Prompt Injection
31. OWASP — LLM Prompt Injection Prevention Cheat Sheet
32. OWASP — AI Agent Security Cheat Sheet
33. OWASP — Top 10 for Agentic Applications 2026
34. NVIDIA NeMo Guardrails — GitHub
35. mem0 — Documentation
36. Weaviate — What Are AI Agents?
37. FinOps Foundation — FinOps for AI Overview

Technical Blogs & Tutorials

38. Vellum — AI Agent Architectures 2026
39. Tungsten Automation — Agentic AI in Production 2026
40. Zylos — CUA State of Computer-Using Agents 2026
41. Zylos — AI Agent Cost Optimization and Token Economics 2026
42. Vectorize.io — AI Memory Systems Comparison 2026
43. Mem0 — State of AI Memory 2026
44. Sierra — τ -bench: A Benchmark for Tool-Agent-User Interaction
45. Evidently AI — AI Agent Evaluation Benchmarks 2026
46. Latitude — LangSmith vs LangFuse vs Helicone: Observability Comparison 2026
47. Laminar — AI Observability with Browser Session Replay
48. WorkOS — MCP 2026: Current State and Future
49. Maxim AI — Best AI Guardrails Platforms in 2026
50. Kili Technology — Guide to AI Agent Benchmarks 2026
51. VentureBeat — AI Agents Fail 1 in 3 Production Deployments
52. Entro Security — NHI Misconfiguration Risks Report
53. Invariant Labs — MCP Tool Poisoning Attacks
54. Anthropic — Model Context Protocol Introduction
55. Developers Digest — Multi-Agent Coordination Patterns 2026
56. MorphLLM — Framework Comparison 2026
57. Evidently AI — LLM Evaluation Guide
58. Langfuse — Tracing and Observability for LLM Apps
59. Helicone — LLM Monitoring
60. Arize Phoenix — LLM Tracing
61. Portkey AI — AI Gateway
62. LlamaIndex — Agentic RAG
63. Simon Willison — LLM Tool Use and Function Calling
64. DeepLearning.AI — Multi-Agent Systems Course
65. Lilian Weng — LLM-Powered Autonomous Agents
66. OpenAI Cookbook — Tool Use Examples
67. Anthropic Cookbook — Tool Use Examples

- 68. [Weights & Biases — Agent Evaluation with Weave](#)
- 70. [Context Engineering Guide 2026](#)
- 72. [Towards Data Science — ReAct Agent Pattern Explained](#)
- 74. [PromptingGuide.AI — Tree of Thoughts](#)
- 76. [AssemblyAI — LLM Agents Tutorial](#)
- 78. [E2B — Code Interpreter for AI Agents](#)

GitHub Repositories & Code Examples

- 79. [LangGraph — GitHub](#)
- 81. [CrewAI — GitHub](#)
- 83. [OpenAI Agents SDK — GitHub](#)
- 85. [NVIDIA NeMo Guardrails — GitHub](#)
- 87. [Mem0 — GitHub](#)
- 89. [MemoryOS — GitHub](#)
- 91. [Microsoft UFO — GitHub](#)
- 93. [AgentBench — GitHub](#)
- 95. [GAIA Benchmark — Hugging Face](#)
- 97. [Instructor — GitHub](#)
- 99. [DSPy — GitHub](#)
- 101. [Toolhouse — GitHub](#)
- 103. [MCP Inspector — GitHub](#)
- 105. [CAMEL — GitHub](#)
- 107. [Agno \(formerly Phidata\) — GitHub](#)
- 109. [Composio — GitHub](#)
- 69. [Braintrust — LLM Evaluation Platform](#)
- 71. [Towards AI — Agentic RAG Patterns](#)
- 73. [Towards Data Science — Chain of Thought Prompting](#)
- 75. [PromptingGuide.AI — ReAct Prompting](#)
- 77. [Composio — Tool Integration for AI Agents](#)
- 80. [AutoGen — GitHub](#)
- 82. [OpenAI Swarm \(experimental\) — GitHub](#)
- 84. [Hugging Face smolagents — GitHub](#)
- 86. [Guardrails AI — GitHub](#)
- 88. [Zep — GitHub](#)
- 90. [browser-use — GitHub](#)
- 92. [WebArena — GitHub](#)
- 94. [SWE-bench — GitHub](#)
- 96. [HAL Leaderboard — GitHub](#)
- 98. [Pydantic AI — GitHub](#)
- 100. [Semantic Kernel — GitHub](#)
- 102. [OpenTelemetry GenAI SIG — GitHub](#)
- 104. [Microsoft Agent Governance Toolkit — GitHub](#)
- 106. [Phidata — GitHub](#)
- 108. [Temporal AI Workflows — GitHub](#)
- 110. [E2B — GitHub](#)

Academic Papers

- 111. [arXiv 2210.03629 — ReAct: Synergizing Reasoning and Acting in LLMs](#)
- 113. [arXiv 2305.10601 — Tree of Thoughts: Deliberate Problem Solving](#)
- 115. [arXiv 2303.17760 — HuggingGPT: Solving AI Tasks with ChatGPT](#)
- 117. [arXiv 2402.05120 — AgentBench: Evaluating LLMs as Agents](#)
- 119. [arXiv 2407.15957 — GAIA: A Benchmark for General AI Assistants](#)
- 121. [arXiv 2406.12925 — \$\tau\$ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains](#)
- 123. [arXiv 2407.01230 — HAL: A Framework for Holistic Evaluation of AI Agents](#)
- 125. [arXiv 2309.07864 — Cognitive Architectures for Language Agents \(CoALA\)](#)
- 127. [arXiv 2304.03442 — Generative Agents: Interactive Simulacra of Human Behavior](#)
- 129. [arXiv 2301.04589 — Toolformer: Language Models Can Teach Themselves to Use Tools](#)
- 131. [arXiv 2404.01349 — LLM Agent Operating System \(AIOS\)](#)
- 133. [arXiv 2412.04093 — From Solo Performance to Symphony: AI Agents in Multi-Agent Systems](#)
- 112. [arXiv 2201.11903 — Chain-of-Thought Prompting Elicits Reasoning in LLMs](#)
- 114. [arXiv 2308.09687 — Graph of Thoughts: Solving Elaborate Problems with LLMs](#)
- 116. [arXiv 2309.02427 — Agents: An Open-source Framework for Autonomous Language Agents](#)
- 118. [arXiv 2311.12983 — JARVIS-1: Open-World Multi-Task Agents with Memory-Augmented Multimodal Language Models](#)
- 120. [arXiv 2310.11667 — SWE-bench: Can Language Models Resolve Real-World GitHub Issues?](#)
- 122. [arXiv 2504.19413 — Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory](#)
- 124. [arXiv 2405.14751 — Observational Scaling Laws and the Predictability of Language Model Performance](#)
- 126. [arXiv 2308.10848 — MetaGPT: Meta Programming for Multi-Agent Collaboration](#)
- 128. [arXiv 2401.05268 — WebArena: A Realistic Web Environment for Building Autonomous Agents](#)
- 130. [arXiv 2406.06608 — SWE-bench Verified](#)
- 132. [arXiv 2410.02107 — OpenAI o1 System Card](#)
- 134. [arXiv 2503.01214 — Computer-Using Agents: Survey and Benchmark Analysis](#)

Video Resources

- 135. [Andrej Karpathy — Intro to Large Language Models \(YouTube\)](#)
- 137. [DeepLearning.AI — Functions, Tools, and Agents with LangChain \(YouTube\)](#)
- 139. [Matt Wolfe — AI Agents in 2026 Overview \(YouTube\)](#)
- 141. [LangChain — LangGraph Tutorial \(YouTube\)](#)
- 136. [Andrew Ng — Agentic AI Design Patterns \(YouTube\)](#)
- 138. [AI Explained — OpenAI o1 and Chain of Thought \(YouTube\)](#)
- 140. [Yannic Kilcher — ReAct Paper Explained \(YouTube\)](#)
- 142. [Microsoft Research — AutoGen Multi-Agent Systems \(YouTube\)](#)

Industry Best Practice Guides & Books

- 143. [Stanford HAI — AI Index Report 2026](#)
- 145. [Google — Agents Companion: Foundational Guide](#)
- 147. [Azure — AI Agent Service Overview](#)
- 149. [Eugene Yan — Patterns for Building LLM-based Systems and Products](#)
- 144. [OWASP — LLM Security Top 10 v2.0](#)
- 146. [AWS — Agentic AI Reference Architecture](#)
- 148. [Chip Huyen — AI Engineering \(O'Reilly, 2025\)](#)
- 150. [Hamel Husain — Your AI Product Needs Evals](#)

- 151. CISA — Guidelines for Secure AI System Development
- 153. McKinsey — The State of AI 2026
- 155. Sequoia Capital — AI Agents Market Map 2026

- 152. Gartner — Agentic AI Hype Cycle 2025
- 154. a16z — The Current State of Agentic AI
- 156. InfoQ — AI Agent Architecture Patterns Guide 2026